

GESTÃO, ORQUESTRAÇÃO E AUTOMAÇÃO EM REDES OPENRAN

Provisionamento e gerenciamento de uma pilha Open RAN utilizando SMO

Parte 2 — implementação, operação e análise dos mecanismos

Integrantes: Charles Siqueira Xavier, Felipe Bartholo Favilla,
Luciano Augusto Campagnoli da Silva, Thiago Bezerra Coellho

De onde partimos

A escolha da Parte 1 e o que a Parte 2 precisava provar

1

Ferramenta escolhida

SMO da O-RAN Software Community: módulo OAM para a interface O1 e Non-RT RIC para a A1, na M-Release de dezembro de 2025.

2

Estratégia recomendada

Começar pelo perfil leve com escopo O1/OAM, validar coleta e configuração, e só então incluir o Non-RT RIC. Cumprida integralmente.

3

O que a Parte 2 exige

Provisionar, gerenciar e acompanhar uma pilha Open RAN — e analisar os mecanismos de gestão, orquestração e operação empregados.

A diferença desta etapa:

A Parte 1 analisou o que a plataforma documenta. A Parte 2 mostra o que ela faz quando colocada em operação — inclusive onde foi preciso divergir da distribuição oficial.

O que foi implantado

Dez imagens oficiais da O-RAN SC e do ONAP, mais um portal próprio

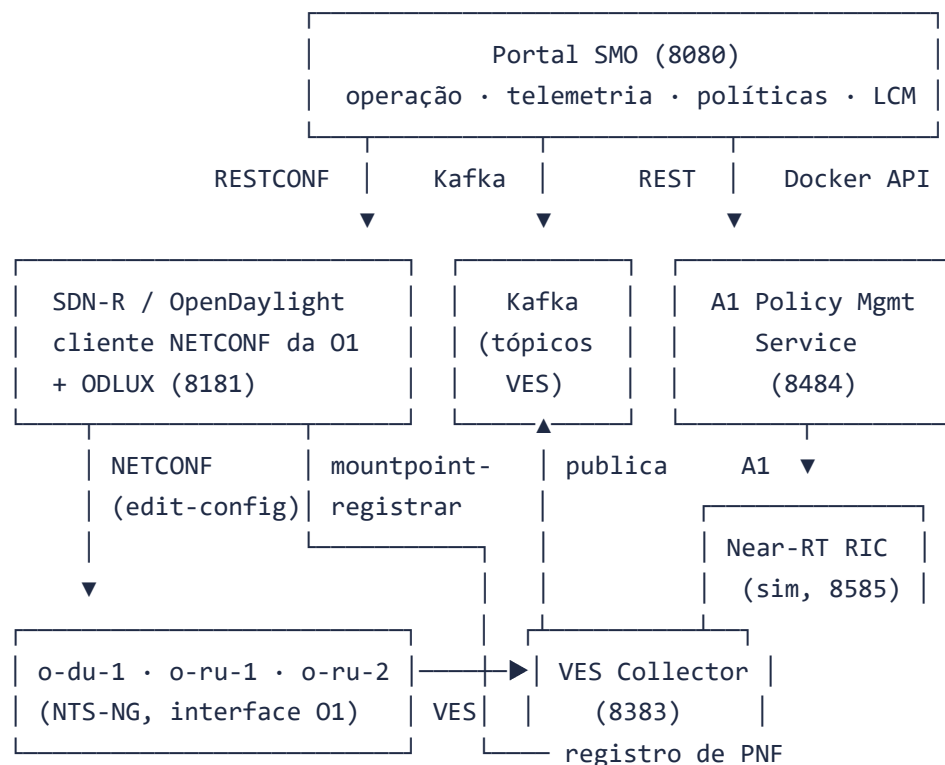
Camada	Serviço	Imagem oficial
SMO / OAM	controller — SDN-R (OpenDaylight), cliente NETCONF da O1	sdnr-image:13.0.1
SMO / OAM	odlux — interface web oficial do controlador	sdnr-web-image:13.0.1
SMO / OAM	ves-collector — ingestão de eventos da O1	vescollector:1.12.5 (ONAP)
Mensageria	kafka + zookeeper — barramento de eventos	strimzi/kafka:0.35.0
Rede gerenciada	o-du-1, o-ru-1, o-ru-2 — funções de rede simuladas	nts-ng-*:1.8.1
Non-RT RIC	a1-pms + near-rt-ric — interface A1	a1-policy-mgmt:2.3.1 · a1-simulator:2.2.0
Operação	smo-portal — portal deste trabalho	construída localmente

Linha divisória:

Tudo que executa funções de SMO é software oficial. O portal apenas consome as APIs desses componentes — não reimplementa nenhuma função de gerência.

Arquitetura implantada

Cinco camadas e os caminhos de dados entre elas



Os três elementos conectaram-se anunciando 33, 84 e 84 módulos YANG, além de 13 capacidades de protocolo cada.

Como um elemento entra sob gestão do SMO

O mecanismo mais característico da solução: descoberta automática

- **1. Registro.** A função de rede sobe e envia um evento VES de registro de PNF ao coletor.
- **2. Publicação.** O coletor valida o evento e o publica no tópico VES_PNFREG_OUTPUT do Kafka.
- **3. Montagem.** O mountpoint-registrar do SDN-R consome o tópico e grava o nó NETCONF com endereço, porta e credenciais.
- **4. Descoberta de capacidade.** O OpenDaylight abre a sessão NETCONF e lê os módulos YANG que o elemento anuncia.
- **5. Provisionamento.** O SMO aplica o perfil de dia 1 pela O1, e a telemetria começa a fluir.

O que isso demonstra:

Nenhuma etapa exige intervenção manual ou conhecimento prévio do elemento: o SMO descobre, monta e configura. É o gerenciamento de configuração da interface O1 na prática.

Provisionamento executado pelo SMO

Os elementos conectam — mas não reportam nada até serem configurados

1

O achado

Os simuladores sobem com os parâmetros de telemetria vazios. Conectam pela O1 e não emitem heartbeat, falha nem medição alguma.

2

O que o modelo diz

O YANG do O-DU é explícito: "para ativar a produção da medição, o SMO precisa criar uma instância de performance-measurement-job".

3

A resposta

O portal aplica um perfil de dia 1 a cada elemento que conecta — cinco edit-config no O-DU, três nos O-RU, cada um com releitura de confirmação.

Seletividade:

Cada item do perfil declara o módulo YANG de que depende; só é aplicado a elementos que anunciam esse módulo. O job de medição é montado a partir das células que o elemento expõe.

Interface 01 — gerenciamento de configuração

RESTCONF ao norte, NETCONF ao sul, mediados pelo SDN-R

Operação na interface norte (RESTCONF)	Operação emitida ao elemento (NETCONF)
GET .../yang-ext:mount/<caminho>?content=config	<get-config> no datastore running
GET ...?content=nonconfig	<get> no datastore operational
PUT .../yang-ext:mount/<caminho>	<edit-config> com substituição do ramo
DELETE .../yang-ext:mount/<caminho>	<edit-config> com operação de remoção

Verificação por ida e volta: alterando o período de heartbeat de um O-RU de 30 para 20 segundos, o controlador respondeu HTTP 204 e a releitura devolveu o novo valor — a escrita chegou ao elemento e foi aplicada, não apenas aceita pelo controlador.

Detalhe que confunde:

HTTP 409 com data-missing não significa módulo desconhecido, mas ramo sem dado gravado. E a leitura de um nó de alto nível pode podar listas de estado que só aparecem no nó filho.

Coleta e barramento de dados

Um caminho único: VES → validação → tópico Kafka por domínio

Tópico	Conteúdo	Quem consome
VES_PNFREG_OUTPUT	registro de funções de rede	mountpoint-registrar e portal
SEC_FAULT_OUTPUT	eventos de falha	portal (gerenciamento de falhas)
SEC_HEARTBEAT_OUTPUT	sinais de vida periódicos	portal (supervisão)
VES_O_RAN_SC_HELLO_WORLD_PM_STREAMING_OUTPUT	medições de desempenho	portal (KPIs)
VES_MEASUREMENT_OUTPUT · VES_FILE_READY_OUTPUT · SEC_3GPP_* · SEC_OTHER	demais domínios	—

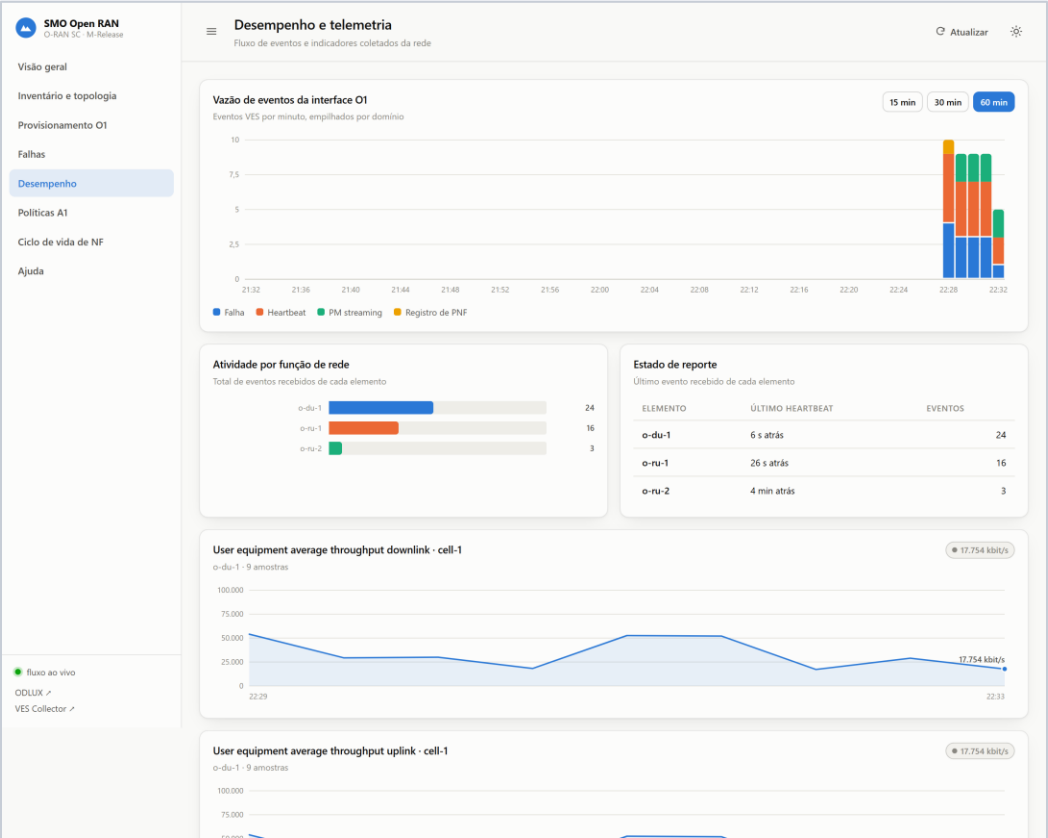
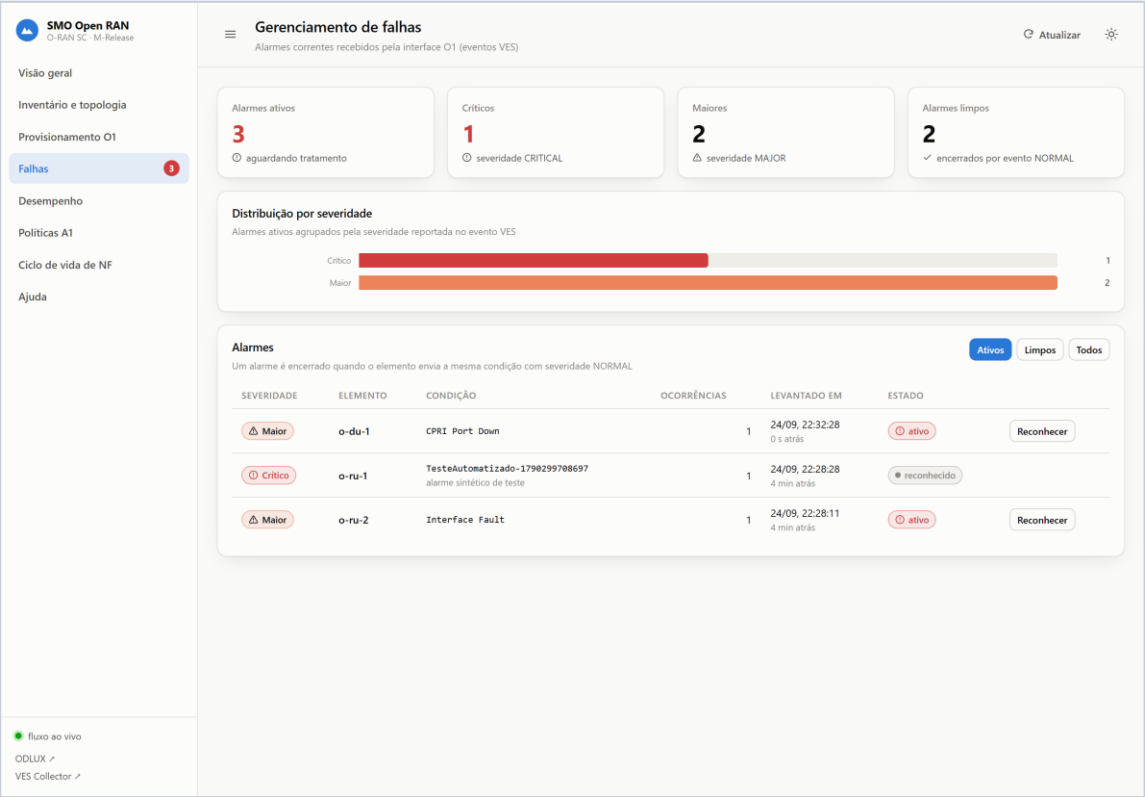
Mudança na M-Release:

O DMaaP Message Router foi abandonado; o coletor publica direto no Kafka. Os arquivos de configuração ainda trazem URLs no formato antigo, mas só o nome do tópico é usado.

Em uma hora de operação: 10 registros de PNF, 271 heartbeats, 174 eventos de falha e 31 relatórios de medição.

Falhas e telemetria

Alarmes correntes e séries de KPI construídos a partir do fluxo VES

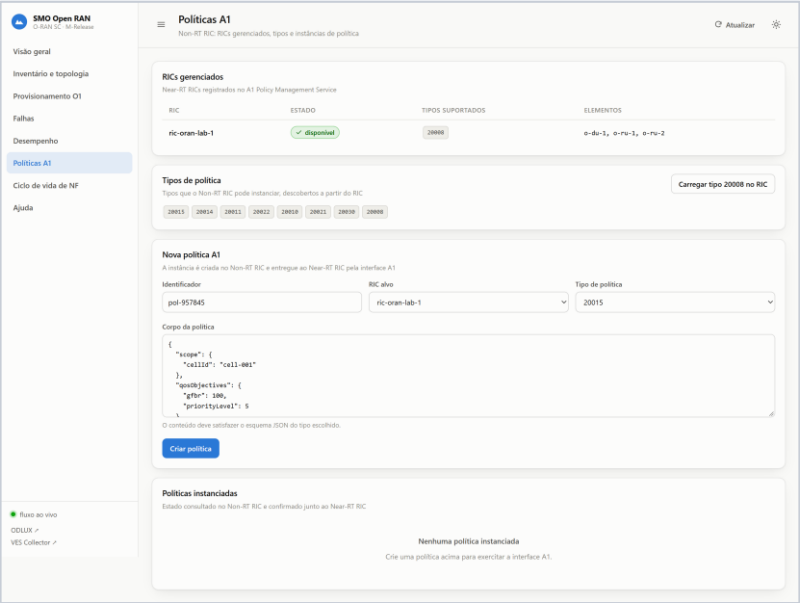


Alarme encerrado automaticamente quando o elemento reenvia a condição com severidade NORMAL. KPIs de vazão por célula a cada 30 s, em kbit/s.

Políticas A1 no Non-RT RIC

Ciclo completo verificado: tipo, instância, entrega e remoção

- **Carga do tipo.** O simulador sobe sem tipos definidos — nada a instanciar até que um tipo seja carregado no RIC.
- **Sincronização.** O A1 Policy Management Service passa a listar o tipo poucos segundos depois.
- **Criação.** Instância criada pelo Non-RT RIC, com resposta HTTP 201.
- **Confirmação no RIC.** A política foi localizada no próprio Near-RT RIC — entrega pela A1, não apenas registro.
- **Remoção.** Ciclo de vida encerrado com HTTP 204.



Detalhe de operação:

O simulador exige o tipo embrulhado em uma estrutura com nome, descrição, identificador e esquema de criação — e não o esquema JSON isolado.

Ciclo de vida de Network Functions

E o acoplamento observável com a interface O1

1

No perfil leve

O ciclo de vida se resume a iniciar, monitorar e encerrar contêineres. Não há orquestrador de funções de rede na distribuição Docker Compose.

2

O que se observa

Ao parar um O-RU, o mountpoint NETCONF deixa o estado connected em segundos e a aresta O1 muda de cor no diagrama de topologia.

3

Ao reiniciar

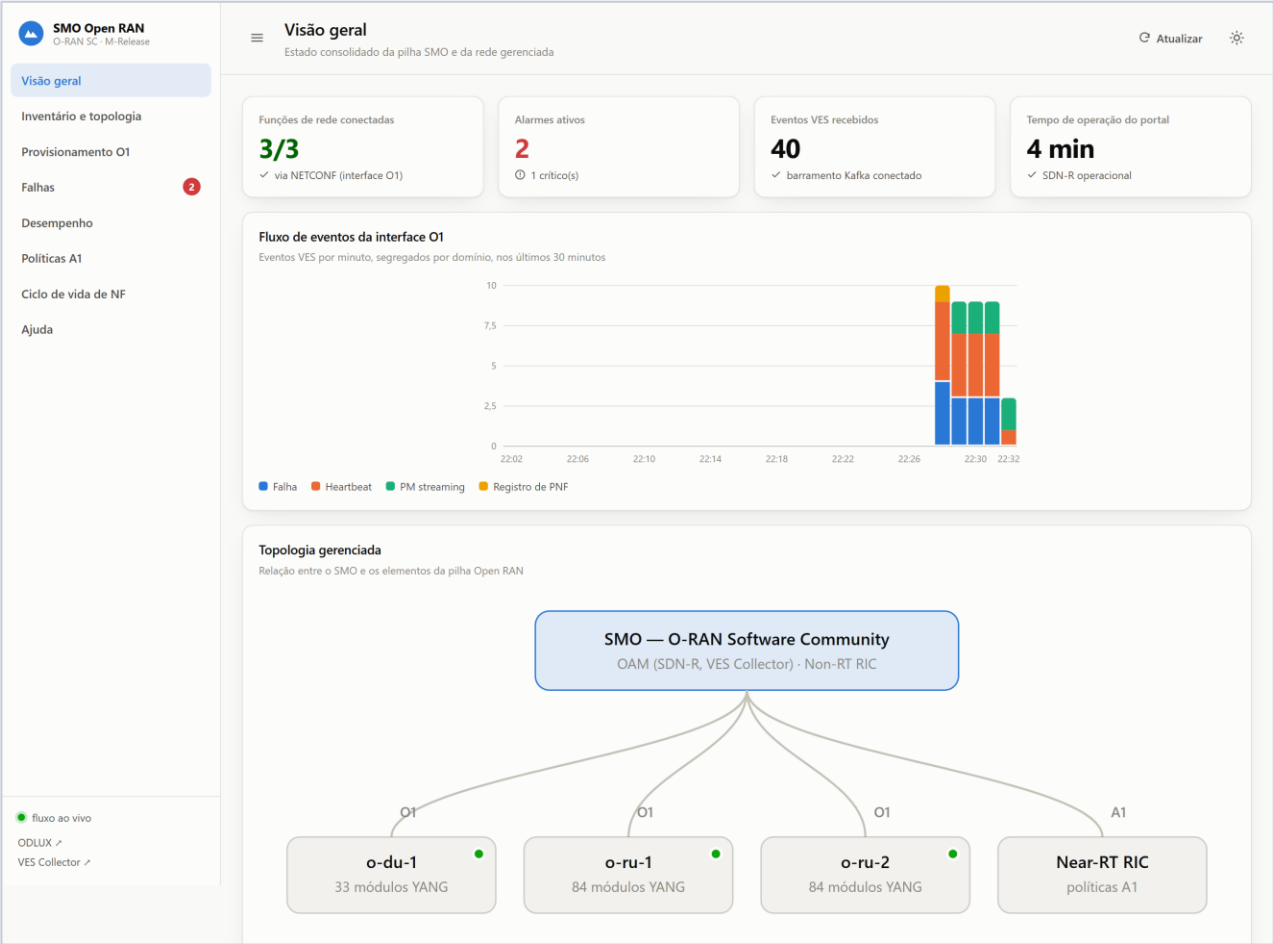
O ciclo completo de registro por VES e remontagem se repete sem ação manual — o SMO mantém um modelo vivo da rede, não um inventário estático.

Perfil completo:

Em Kubernetes o ciclo de vida passaria a Helm; sobre O-Cloud, à função O2dms. Ambos fora do escopo desta implantação, como a Parte 1 já antecipava.

O portal de operação

Oito telas, uma para cada mecanismo do objeto de estudo



- Visão geral · Inventário e topologia
- Provisionamento O1 · Falhas
- Desempenho · Políticas A1
- Ciclo de vida de NF · Ajuda

Backend com duas dependências; frontend sem etapa de compilação e sem CDN — funciona offline.

Paleta de séries validada para daltonismo e contraste nos temas claro e escuro. Cores de estado sempre acompanhadas de ícone e rótulo.

Sem banco de dados: os dados persistentes vivem nos componentes oficiais.

O que precisou divergir do compose oficial

Quatro condições para que a pilha funcionasse — e o que elas revelam

Problema encontrado	Causa	Solução adotada
Controlador sempre "unhealthy"	A verificação oficial usa /ready, endpoint da camada SDNC do ONAP, ausente em modo SDNRONLY	Verificar o próprio RESTCONF da topologia
Elementos montados sem estado	NETCONF Call Home e registro por VES ativos ao mesmo tempo montam o dispositivo duas vezes	Usar apenas o registro por VES, o fluxo canônico da OSC
O-RU invisível para o SMO	Falha de segmentação do simulador ao validar o-ran-usermgmt durante datastore-populate	Desativar essa etapa nos O-RU
A1 PMS sempre "unhealthy"	A imagem não traz curl nem wget, ferramentas que sua própria verificação presumia	Reescrever a verificação com /dev/tcp do bash

O padrão:

Nenhum era defeito funcional de um módulo. Todos eram problemas de montagem — e é neles que o tempo de uma implantação real se concentra.

Verificação end-to-end

Roteiro executável, versionado junto ao código

- **Saúde da pilha.** Nenhum serviço em estado unhealthy.
- **O1 — descoberta.** 3 elementos montados, com 33 e 84 módulos YANG anunciados.
- **O1 — configuração.** Leitura HTTP 200; escrita HTTP 204 confirmada por releitura do datastore.
- **O1 — telemetria.** 8 tópicos VES criados pelo coletor e consumidos pelo portal.
- **Falhas.** Evento injetado aceito com HTTP 202 e visível como alarme crítico no portal.
- **A1.** Política criada (201), confirmada no Near-RT RIC e removida (204).
- **Ciclo de vida.** Parada derruba o mountpoint O1; reinício restabelece por registro automático.

Além do roteiro, 87 testes automatizados sobre o portal — 60 de API e 27 de interface num navegador real, sem simulação de dependências.

Resultado:

Treze passos concluídos, nenhuma falha. A suíte encontrou dois defeitos que o uso manual não revelara, ambos em nomes de campo de APIs de terceiros.

Frente ao modelo de referência de SMO

A avaliação da Parte 1, agora com evidência de execução

Componente	Parte 1	Observado na implementação
OAM	implementado	Confirmado. Núcleo maduro: configuração, coleta e falhas funcionam pela O1.
Non-RT RIC	implementado	Confirmado quanto a políticas A1. O rApp Manager exige o perfil Kubernetes.
TE&IV	implementado	Parcialmente observado: topologia e capacidades vêm do controlador; o projeto TEIV não integra o perfil leve.
SA	parcial	Confirmado. Os dados existem; o fechamento da malha depende de uma rApp.
NFO	parcial	Confirmado. No perfil leve não há orquestrador de NF, apenas o ciclo de vida dos contêineres.
SO · FOCOM	não implementado	Não observados. FOCOM depende do O-Cloud, que exige hardware dedicado.

A nuance que a documentação não antecipa:

A distância entre "a plataforma implementa a função" e "a função está operacional após a instalação" é maior do que a documentação sugere.

Conclusões

Três leituras sobre os mecanismos da solução

1

O núcleo é maduro

A cadeia que vai do registro por evento VES até a configuração por NETCONF opera sem intervenção manual, e o modelo de rede reage em segundos a mudanças de estado.

2

A fronteira está adiante

Elementos conectam sem reportar nada; não há KPI sem job de medição; não há política sem tipo carregado. Reconhecer essa fronteira é entender o que é operar um SMO.

3

O custo está na integração

Os quatro problemas encontrados eram de montagem, não de função. É neles que o tempo de uma implantação real se concentra.

Entregue e reproduzível:

Pilha completa, portal de operação, scripts de verificação e relatório publicados em github.com/felipefavilla/smo-openran-parte2

Referências

Fontes oficiais e repositório do trabalho

- O-RAN SOFTWARE COMMUNITY. OAM — SMO Deployment and Configuration. docs.o-ran-sc.org
- O-RAN SOFTWARE COMMUNITY. Repositórios oam, sim-o1-interface e nonrtric. github.com/o-ran-sc
- O-RAN ALLIANCE. O-RAN Architecture Description. o-ran.org
- IETF. RFC 6241 — NETCONF. RFC 8040 — RESTCONF Protocol.
- ONAP. VES Collector — DCAE Gen2. docs.onap.org
- 3GPP. TS 28.552 — Management and orchestration; 5G performance measurements.
- FAVILLA, F. B. et al. Relatório — Parte 1: pesquisa, comparação e seleção de plataforma de SMO. CPQD, 2026.

Repositório:

github.com/felipefavilla/smo-openran-parte2